

Laboratory Work 2

Normalization and DDL

Course: Databases

Student: BEKTURSYN ADINA MURATKYZY

Date: September 2026

Normalization of Booking Receipt

This section normalizes the Booking_receipt relation to Third Normal Form and implements the resulting tables. The design assumes that one booking is for one flight and may contain several passengers, each with one assigned seat.

Task 1 Initial Normal Form Violation Analysis

Original relation: Booking_receipt(booking_id, passenger_full_name, passenger_passport_number, flight_number, departure_airport_name, departure_city, arrival_airport_name, arrival_city, airline_name, seat_numbers, ticket_price).

The relation violates First Normal Form (1NF) because seat_numbers can contain several values in one cell, for example "12A, 12B". A 1NF attribute must hold a single atomic value. The remaining attributes may be represented by one value per row. In particular, passenger_full_name is treated as an atomic string for this task; splitting it into first and last name is optional and is not required for 1NF.

Since the relation is not in 1NF, it cannot be in 2NF or 3NF either.

Tasks 2 and 3 Normalization Steps and Functional Dependencies

Functional Dependencies Used

booking_id determines flight_number, ticket_price

flight_number determines departure_airport_name, arrival_airport_name, airline_name

departure_airport_name determines departure_city

arrival_airport_name determines arrival_city

passenger_passport_number determines passenger_full_name

(booking_id, passenger_passport_number) determines seat_number

Step 1 Conversion to 1NF

Replace the comma-separated seat_numbers value with one row for each passenger and seat assignment. The primary key is booking_id and passenger_passport_number. This key is appropriate because a passenger appears at most once in one booking.

Booking_receipt_1NF (booking_id, passenger_passport_number, passenger_full_name, seat_number, flight_number, departure_airport_name, departure_city, arrival_airport_name, arrival_city, airline_name, ticket_price)

Primary key: booking_id, passenger_passport_number

For example, booking 1001 with seats 12A and 12B becomes two rows, rather than one row containing "12A, 12B".

Step 2 Conversion to 2NF

The 1NF relation has a composite key. Attributes flight_number and ticket_price depend only on booking_id. Passenger_full_name depends only on passenger_passport_number. These are partial dependencies, so they are removed into separate tables.

Booking_2NF (booking_id, flight_number, departure_airport_name, departure_city, arrival_airport_name, arrival_city, airline_name, ticket_price) Primary key: booking_id

Passenger_2NF (passenger_passport_number, passenger_full_name) Primary key: passenger_passport_number

Booking_passenger_2NF (booking_id, passenger_passport_number, seat_number) Primary key: booking_id, passenger_passport_number Foreign keys: booking_id references Booking_2NF(booking_id); passenger_passport_number references Passenger_2NF(passenger_passport_number)

Step 3 Conversion to 3NF

Booking_2NF still contains transitive dependencies: booking_id determines flight_number which determines departure_airport_name, arrival_airport_name, airline_name; departure_airport_name determines departure_city; and arrival_airport_name determines arrival_city. Decompose these attributes into Flight, Airport and Airline. The final 3NF relations are as follows:

Passenger (passenger_passport_number, passenger_full_name) Primary key: passenger_passport_number

Airport (airport_name, city) Primary key: airport_name

Airline (airline_name) Primary key: airline_name

Flight (flight_number, departure_airport_name, arrival_airport_name, airline_name) Primary key: flight_number Foreign keys: departure_airport_name references Airport(airport_name); arrival_airport_name references Airport(airport_name); airline_name references Airline(airline_name)

Booking (booking_id, flight_number, ticket_price) Primary key: booking_id Foreign keys: flight_number references Flight(flight_number)

Booking_passenger (booking_id, passenger_passport_number, seat_number) Primary key:
booking_id, passenger_passport_number Foreign keys: booking_id references
Booking(booking_id); passenger_passport_number references
Passenger(passenger_passport_number)

Task 4 3NF Verification for the DDL Flights Table

Current primary key: flight_id.

Functional dependency: flight_id determines sch_departure_time, sch_arrival_time,
departing_airport_id, arriving_airport_id, departing_gate, arriving_gate, airline_id,
act_departure_time, act_arrival_time, created_at, updated_at.

The Flights table is in 3NF. Its primary key has one attribute, so partial dependencies are impossible. The non-key attributes describe the flight and depend on flight_id.
departing_airport_id, arriving_airport_id and airline_id are foreign keys only; the airport city/name and airline name/country are stored in Airport and Airline, not in Flights. Therefore there is no transitive dependency between non-key attributes within Flights.

Task 5. Implementation of 3NF tables

lab2

lab2_ddl.sql • PostgreSQL Get Started lab2_normalization.sql X

lab2_normalization.sql

localhost | postgres | schema: session default

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTGRESQL QUERY RESULTS

Results Messages

localhost | postgres | schema: session default

	passenger_passport_number	passenger_full_name
1	N123456	John Doe
2	N234567	Jane Smith
3	N345678	Alex Brown
4	N456789	Emily Davis
5	N567890	Michael Wilson
6	N678901	Sarah Miller

	airport_name	city
1	Charles de Gaulle	Paris
2	Dubai International	Dubai
3	Heathrow	London
4	JFK International	New York
5	Tokyo Haneda	Tokyo

	airline_name
1	Air France
2	British Airways
3	Delta Air Lines
4	Emirates
5	Japan Airlines

	flight_number	departure_airport_name	arrival_airport_name	airline_name
1	AF404	Charles de Gaulle	Dubai International	Air France
2	BA202	Heathrow	Tokyo Haneda	British Airways
3	DL101	JFK International	Heathrow	Delta Air Lines
4	EK505	Dubai International	JFK International	Emirates
5	JL303	Tokyo Haneda	Charles de Gaulle	Japan Airlines

	booking_id	flight_number	ticket_price
1	1001	DL101	900.00
2	1002	BA202	600.00
3	1003	JL303	750.00
4	1004	AF404	520.00
5	1005	EK505	890.00

	booking_id	passenger_passport_number	seat_number
1	1001	N123456	12A
2	1001	N234567	12B
3	1002	N345678	14C
4	1003	N456789	02F
5	1004	N567890	18D
6	1005	N678901	01A

OUTLINE

TIMELINE

Select Postgres Server

Ln 1

←→

lab2_ddl.sql • PostgreSQL Get Started lab2_normalization.sql X

lab2_normalization.sql

lab2

lab2_ddl.sql

lab2_normalization...

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

71

72

73

74

75

76

77

78

79

80

81

82

83

84

85

86

87

88

89

90

91

92

93

94

95

96

97

98

99

100

101

102

103

104

105

106

107

108

109

110

111

112

```
localhost | postgres | schema: session default
1 CREATE SCHEMA IF NOT EXISTS lab2_normalization;
2 SET search_path TO lab2_normalization;
3
4 -- 3NF tables
5 CREATE TABLE passengers (
6     passenger_passport_number VARCHAR(20) PRIMARY KEY,
7     passenger_full_name VARCHAR(100) NOT NULL
8 );
9
10 CREATE TABLE airports (
11     airport_name VARCHAR(50) PRIMARY KEY,
12     city VARCHAR(50) NOT NULL
13 );
14
15 CREATE TABLE airlines (
16     airline_name VARCHAR(50) PRIMARY KEY
17 );
18
19 CREATE TABLE flights (
20     flight_number VARCHAR(20) PRIMARY KEY,
21     departure_airport_name VARCHAR(50) NOT NULL,
22     arrival_airport_name VARCHAR(50) NOT NULL,
23     airline_name VARCHAR(50) NOT NULL,
24     CONSTRAINT fk_flights_departure_airport
25         FOREIGN KEY (departure_airport_name) REFERENCES airports(airport_name),
26     CONSTRAINT fk_flights_arrival_airport
27         FOREIGN KEY (arrival_airport_name) REFERENCES airports(airport_name),
28     CONSTRAINT fk_flights_airline
29         FOREIGN KEY (airline_name) REFERENCES airlines(airline_name)
30 );
31
32 CREATE TABLE bookings (
33     booking_id INT PRIMARY KEY,
34     flight_number VARCHAR(20) NOT NULL,
35     ticket_price DECIMAL(7,2) NOT NULL,
36     CONSTRAINT fk_bookings_flight
37         FOREIGN KEY (flight_number) REFERENCES flights(flight_number)
38 );
39
40 CREATE TABLE booking_passengers (
41     booking_id INT NOT NULL,
42     passenger_passport_number VARCHAR(20) NOT NULL,
43     seat_number VARCHAR(10) NOT NULL,
44     CONSTRAINT pk_booking_passengers
45         PRIMARY KEY (booking_id, passenger_passport_number),
46     CONSTRAINT fk_booking_passengers_booking
47         FOREIGN KEY (booking_id) REFERENCES bookings(booking_id),
48     CONSTRAINT fk_booking_passengers_passenger
49         FOREIGN KEY (passenger_passport_number)
50             REFERENCES passengers(passenger_passport_number),
51     CONSTRAINT uq_booking_passengers_seat
52         UNIQUE (booking_id, seat_number)
53 );
54
55 -- The following data is split from a denormalized booking-receipt dataset.
56 -- Booking 1001 demonstrates the original repeating group: seats 12A and 12B.
57
58 INSERT INTO passengers (passenger_passport_number, passenger_full_name) VALUES
59     ('N123456', 'John Doe'),
60     ('N234567', 'Jane Smith'),
61     ('N345678', 'Alex Brown'),
62     ('N456789', 'Emily Davis'),
63     ('N567890', 'Michael Wilson'),
64     ('N678901', 'Sarah Miller');
65
66 INSERT INTO airports (airport_name, city) VALUES
67     ('JFK International', 'New York'),
68     ('Heathrow', 'London'),
69     ('Tokyo Haneda', 'Tokyo'),
70     ('Charles de Gaulle', 'Paris'),
71     ('Dubai International', 'Dubai');
72
73 INSERT INTO airlines (airline_name) VALUES
74     ('Delta Air Lines'),
75     ('British Airways'),
76     ('Japan Airlines'),
77     ('Air France'),
78     ('Emirates');
79
80 INSERT INTO flights
81     (flight_number, departure_airport_name, arrival_airport_name, airline_name)
82 VALUES
83     ('JL101', 'JFK International', 'Heathrow', 'Delta Air Lines'),
84     ('BA202', 'Heathrow', 'Tokyo Haneda', 'British Airways'),
85     ('JL303', 'Tokyo Haneda', 'Charles de Gaulle', 'Japan Airlines'),
86     ('AF404', 'Charles de Gaulle', 'Dubai International', 'Air France'),
87     ('EK505', 'Dubai International', 'JFK International', 'Emirates');
88
89 INSERT INTO bookings (booking_id, flight_number, ticket_price) VALUES
90     (1001, 'JL101', 990.00),
91     (1002, 'BA202', 600.00),
92     (1003, 'JL303', 750.00),
93     (1004, 'AF404', 520.00),
94     (1005, 'EK505', 890.00);
95
96 INSERT INTO booking_passengers
97     (booking_id, passenger_passport_number, seat_number)
98 VALUES
99     (1001, 'N123456', '12A'),
100     (1001, 'N234567', '12B'),
101     (1002, 'N345678', '14C'),
102     (1003, 'N456789', '02F'),
103     (1004, 'N567890', '10D'),
104     (1005, 'N678901', '01A');
105
106 -- Queries to run for screenshots after executing the script.
107 SELECT * FROM passengers ORDER BY passenger_passport_number;
108 SELECT * FROM airports ORDER BY airport_name;
109 SELECT * FROM airlines ORDER BY airline_name;
110 SELECT * FROM flights ORDER BY flight_number;
111 SELECT * FROM bookings ORDER BY booking_id;
112 SELECT * FROM booking_passengers ORDER BY booking_id, seat_number;
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

POSTGRES SQL QUERY RESULTS

OUTLINE

TIMELINE

Results

Messages

Select Postgres Server

```

9      airline_code VARCHAR(30) NOT NULL,
10     airline_name VARCHAR(50) NOT NULL,
11     airline_country VARCHAR(50) NOT NULL,
12     created_at TIMESTAMP NOT NULL,
13     updated_at TIMESTAMP NOT NULL,
14     info VARCHAR(50) NOT NULL
15 );
16
17 CREATE TABLE Airport (
18     airport_id INT PRIMARY KEY,
19     airport_name VARCHAR(50) NOT NULL,

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTGRESQL QUERY RESULTS

Messages

```

9:45:19 PM      Started executing query at Line 1
NOTICE: drop cascades to 11 other objects
DROP SCHEMA
9:45:19 PM      Started executing query at Line 2
CREATE SCHEMA
9:45:19 PM      Started executing query at Line 3
SET
9:45:19 PM      Started executing query at Line 5
CREATE TABLE
9:45:19 PM      Started executing query at Line 17
CREATE TABLE
9:45:19 PM      Started executing query at Line 27
CREATE TABLE
9:45:19 PM      Started executing query at Line 40
CREATE TABLE
9:45:19 PM      Started executing query at Line 49
CREATE TABLE
9:45:19 PM      Started executing query at Line 57
CREATE TABLE
9:45:19 PM      Started executing query at Line 66
CREATE TABLE
9:45:19 PM      Started executing query at Line 81
CREATE TABLE
9:45:19 PM      Started executing query at Line 92
CREATE TABLE
9:45:19 PM      Started executing query at Line 100
CREATE TABLE
9:45:19 PM      Started executing query at Line 108
ALTER TABLE
9:45:19 PM      Started executing query at Line 111
ALTER TABLE
9:45:19 PM      Started executing query at Line 114
ALTER TABLE
9:45:19 PM      Started executing query at Line 118
ALTER TABLE
9:45:19 PM      Started executing query at Line 122
ALTER TABLE
9:45:19 PM      Started executing query at Line 130
ALTER TABLE
9:45:19 PM      Started executing query at Line 135
ALTER TABLE
9:45:19 PM      Started executing query at Line 140
ALTER TABLE
9:45:19 PM      Started executing query at Line 146
ALTER TABLE
9:45:19 PM      Started executing query at Line 151
ALTER TABLE
9:45:19 PM      Started executing query at Line 156
ALTER TABLE
9:45:19 PM      Started executing query at Line 161
ALTER TABLE
9:45:19 PM      Started executing query at Line 167
ALTER TABLE
9:45:19 PM      Started executing query at Line 173
ALTER TABLE
9:45:19 PM      Started executing query at Line 179
ALTER TABLE
9:45:19 PM      Started executing query at Line 184
ALTER TABLE
Total execution time: 00:00:00.026

```